

Optimization

3.3.2 Newton's method

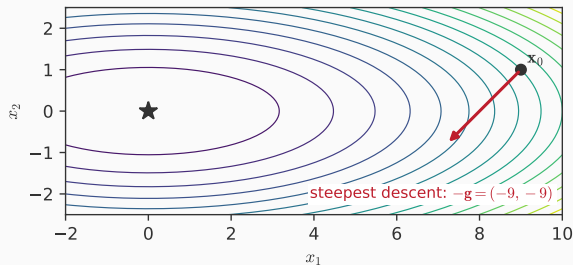
José Firmino Aguilar Madeira

Can we use the curvature?

The question left by 3.3.1. The same quadratic: $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\mathbf{x}_0 = (9, 1)$,

$$\mathbf{g}(\mathbf{x}) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix}, \quad \mathbf{g}_0 = (9, 9).$$

Steepest descent: $\mathbf{d} = -\mathbf{g}_0 = (-9, -9)$
points at the “wall”: 74 iterations.



Can we use the curvature?

The question left by 3.3.1. The same quadratic: $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 9x_2^2)$, $\mathbf{x}_0 = (9, 1)$,

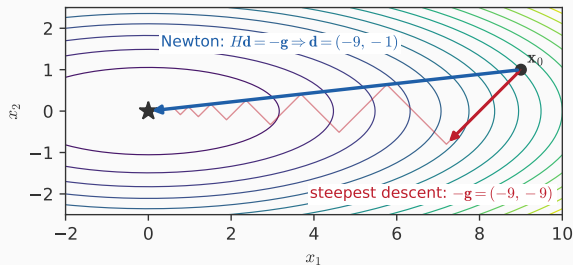
$$\mathbf{g}(\mathbf{x}) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix}, \quad \mathbf{g}_0 = (9, 9).$$

Steepest descent: $\mathbf{d} = -\mathbf{g}_0 = (-9, -9)$
points at the “wall”: 74 iterations.

Newton: solve $H\mathbf{d} = -\mathbf{g}_0$:

$$\mathbf{d}_0 = (-9, -1), \quad \mathbf{x}_1 = \mathbf{x}_0 + \mathbf{d}_0 = (0, 0).$$

One iteration.



Can we use the curvature?

The question left by 3.3.1. The same quadratic: $f(\mathbf{x}) = \frac{1}{2}(\mathbf{x}_1^2 + 9\mathbf{x}_2^2)$, $\mathbf{x}_0 = (9, 1)$,

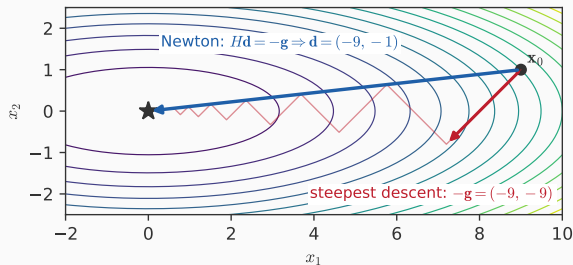
$$\mathbf{g}(\mathbf{x}) = \begin{pmatrix} \mathbf{x}_1 \\ 9\mathbf{x}_2 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix}, \quad \mathbf{g}_0 = (9, 9).$$

Steepest descent: $\mathbf{d} = -\mathbf{g}_0 = (-9, -9)$
points at the “wall”: 74 iterations.

Newton: solve $H\mathbf{d} = -\mathbf{g}_0$:

$$\mathbf{d}_0 = (-9, -1), \quad \mathbf{x}_1 = \mathbf{x}_0 + \mathbf{d}_0 = (0, 0).$$

One iteration.



Why does Newton “know” how to go straight to the minimum?

Where does Newton come from? The Taylor quadratic

At $\mathbf{x}_k$, approximate f by its Taylor quadratic (as in 2.5, now in $\mathbb{R}^n$):

$$f(\mathbf{x}_k + \mathbf{s}) \approx q(\mathbf{s}) = f(\mathbf{x}_k) + \mathbf{g}_k^T \mathbf{s} + \frac{1}{2} \mathbf{s}^T H_k \mathbf{s}, \quad H_k = \nabla^2 f(\mathbf{x}_k).$$

We look for the minimum of the **model**, not directly of f : $\nabla q(\mathbf{s}) = \mathbf{g}_k + H_k \mathbf{s} = \mathbf{0}$, hence

$$\boxed{H_k \mathbf{d}_k = -\mathbf{g}_k} \quad (\text{formally } \mathbf{d}_k = -H_k^{-1} \mathbf{g}_k), \quad \mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{d}_k.$$

- On the computer H_k^{-1} is **never computed**: the system $H_k \mathbf{d}_k = -\mathbf{g}_k$ is solved (Cholesky, Gaussian elimination).
- If f is quadratic with $H \succ 0$, the model is *exact*: the minimum of the model is the minimum of f — **one iteration**, whatever $\mathbf{x}_0$. This is what we saw on the previous page.
- In general the model is only good near $\mathbf{x}_k$: hence the questions on the following pages.

Steepest descent vs. Newton: slope vs. curvature

	Steepest descent (3.3.1)	Newton (3.3.2)
uses	$\mathbf{g}$	$\mathbf{g}$ and H
direction	$\mathbf{d} = -\mathbf{g}$	$H\mathbf{d} = -\mathbf{g}$
geometry	slope	slope + curvature
SPD quadratic	zigzag (linear)	1 iteration
cost per iteration	low	high (Hessian + system)

From 3.3 (“From 3.1. . .”): $\mathbf{d} = -M\mathbf{g}$ with $M \succ 0$ is a descent direction; steepest descent uses $M = I$; when $H \succ 0$, Newton uses $M = H^{-1} \succ 0$.

H^{-1} rescales and rotates the gradient according to the curvature.

In the example: $H^{-1} = \text{diag}(1, \frac{1}{9})$ shrinks the x_2 component (the direction of strong curvature) nine times: $(-9, -9) \rightarrow (-9, -1)$.

When is the Newton direction a descent direction?

$$\mathbf{g}_k^T \mathbf{d}_k = -\mathbf{g}_k^T H_k^{-1} \mathbf{g}_k.$$

- If $H_k \succ 0$ (then $H_k^{-1} \succ 0$): $\mathbf{g}_k^T \mathbf{d}_k < 0$ for every $\mathbf{g}_k \neq \mathbf{0}$ — descent guaranteed.
- If H_k is indefinite or negative definite: there is no guarantee of descent.
- Near a nondegenerate strict minimum ($H(\mathbf{x}^*) \succ 0$), by continuity $H_k \succ 0$: the guarantee is local.
(A strict minimum can have singular $H(\mathbf{x}^*)$: $f(x) = x^4$ at 0.)

What can actually happen when H_k is not positive definite? — coming next, on Himmelblau.

Pure Newton: the algorithm

Pure Newton: $\mathbf{x}_0$, ε_g , $k_{\max}$

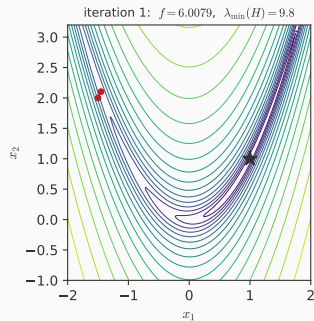
```
1: for  $k = 0, 1, \dots, k_{\max}$  do
2:    $\mathbf{g}_k \leftarrow \nabla f(\mathbf{x}_k)$ ; if  $\|\mathbf{g}_k\| \leq \varepsilon_g$  return  $\mathbf{x}_k$ 
3:    $H_k \leftarrow \nabla^2 f(\mathbf{x}_k)$ 
4:   solve  $H_k \mathbf{d}_k = -\mathbf{g}_k$ 
5:    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \mathbf{d}_k$   $\triangleright$  full step,  $\alpha = 1$ 
6: end for
```

- No line search: the model itself says how far to go.
- **Local quadratic** convergence, under regularity and with $H(\mathbf{x}^*) \succ 0$: near the solution, the number of correct digits tends to double at each iteration.
- SPD quadratic: one iteration, from any $\mathbf{x}_0$.
- Far from the solution: **no guarantee** — neither of descent nor of convergence.

Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

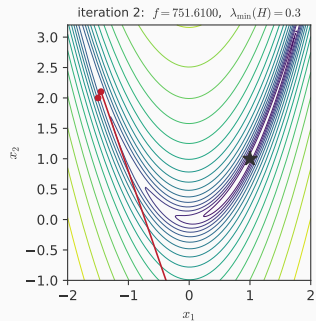
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$



Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

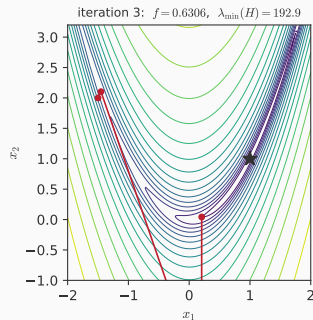
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$



Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

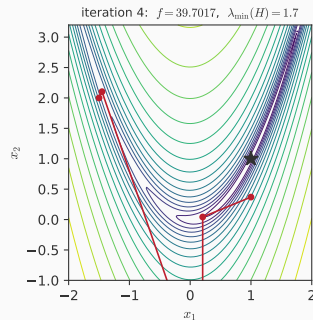
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$
3	$(0.2059, 0.0424)$	0.630622	$1.59 \cdot 10^0$



Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

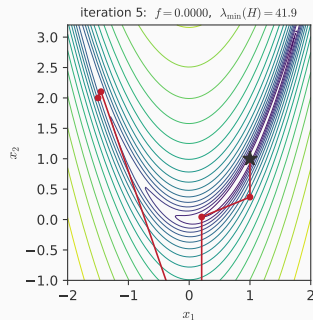
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$
3	$(0.2059, 0.0424)$	0.630622	$1.59 \cdot 10^0$
4	$(0.9997, 0.3692)$	39.701728	$2.82 \cdot 10^2$



Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

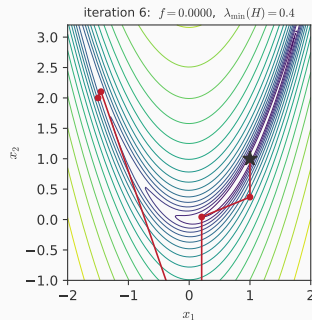
k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$
3	$(0.2059, 0.0424)$	0.630622	$1.59 \cdot 10^0$
4	$(0.9997, 0.3692)$	39.701728	$2.82 \cdot 10^2$
5	$(0.9997, 0.9993)$	$1.09 \cdot 10^{-7}$	$6.61 \cdot 10^{-4}$



Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$
3	$(0.2059, 0.0424)$	0.630622	$1.59 \cdot 10^0$
4	$(0.9997, 0.3692)$	39.701728	$2.82 \cdot 10^2$
5	$(0.9997, 0.9993)$	$1.09 \cdot 10^{-7}$	$6.61 \cdot 10^{-4}$
6	$(1.0000, 1.0000)$	$1.20 \cdot 10^{-12}$	$4.89 \cdot 10^{-5}$

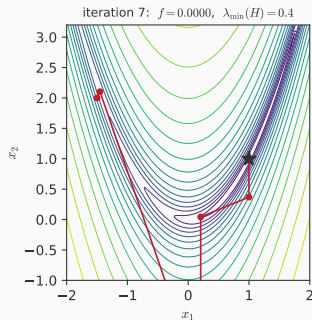


Rosenbrock: very fast, but not monotone

Pure Newton from $(-1.5, 2)$.

k	$\mathbf{x}_k$	$f(\mathbf{x}_k)$	$\ \nabla f\ $
0	$(-1.5000, 2.0000)$	12.500000	$1.63 \cdot 10^2$
1	$(-1.4510, 2.1029)$	6.007882	$6.31 \cdot 10^0$
2	$(0.2044, -2.6986)$	751.610005	$5.92 \cdot 10^2$
3	$(0.2059, 0.0424)$	0.630622	$1.59 \cdot 10^0$
4	$(0.9997, 0.3692)$	39.701728	$2.82 \cdot 10^2$
5	$(0.9997, 0.9993)$	$1.09 \cdot 10^{-7}$	$6.61 \cdot 10^{-4}$
6	$(1.0000, 1.0000)$	$1.20 \cdot 10^{-12}$	$4.89 \cdot 10^{-5}$
7	$(1.0000, 1.0000)$	0	0

Along the way, f rises to 751 and to 40: the full step is **not monotone** — and even so it converges, with quadratic acceleration at the end.

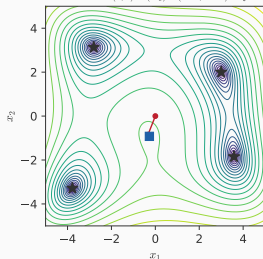


Steepest descent: 3000 it., still $f \approx 2.4 \cdot 10^{-4}$.
Pure Newton: **7 iterations**, $f < 10^{-16}$.

But comparing iterations is not enough: one Newton iteration is much more expensive (Hessian + system).

Newton seeks $\mathbf{g} = \mathbf{0}$ — not necessarily minima

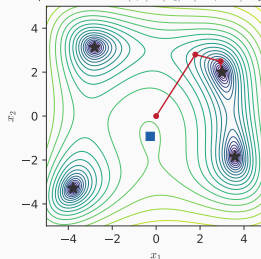
Pure Newton from $(0,0)$: $\lambda(H_0) = (-42, -26) \rightarrow f = 181.617$



Himmelblau from $(0,0)$: $\lambda(H_0) = (-42, -26)$, H_0 **negative definite**. The Newton direction points **uphill**; the pure method converges in 4 iterations — to the **local maximum** $(-0.27, -0.92)$, $f = 181.6$.

The damped method detects $\mathbf{g}^T \mathbf{d} \geq 0$, uses $-\mathbf{g}$ with line search and reaches the minimum $(3, 2)$ in 5 iterations.

Damped Newton from $(0,0)$: $\lambda(H_0) = (-42, -26) \rightarrow f = 0.000$



Newton seeks $\nabla f(\mathbf{x}) = \mathbf{0}$

Pure Newton solves the first-order conditions — and $\nabla f = \mathbf{0}$ includes **minima, maxima and saddles** (3.1: stationary $\neq$ minimum). Only if $H_k \succ 0$: *unique* model minimum and a descent direction.

Damped Newton: protect when far, let it run when near

1. compute $\mathbf{g}_k$, H_k ;
2. test $H_k \succ 0$ (Cholesky);
3. if it fails: modify, $H_k + \mu I$ with sufficiently large $\mu > 0$, or use $-\mathbf{g}_k$;
4. solve $H_k \mathbf{d}_k = -\mathbf{g}_k$;
5. ensure $\mathbf{g}_k^T \mathbf{d}_k < 0$ (otherwise, $\mathbf{d}_k = -\mathbf{g}_k$);
6. line search: $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{d}_k$, with sufficient decrease (Armijo/Wolfe), starting at $\alpha = 1$.

far: protect

near: $\alpha_k \rightarrow 1$, let Newton run

Property (globalized Newton)

With an Armijo/Wolfe-type line search starting at $\alpha = 1$, under suitable conditions, once the iterates are sufficiently close to the solution the full step $\alpha_k = 1$ is accepted — and quadratic convergence is recovered.

In the experiments of this course (safeguard of step 5 + very accurate one-dimensional minimization): Rosenbrock, pure 7 iterations (not monotone), damped 14 (monotone); the last line minimizers were 0.91, 0.98, 1.00 — a numerical illustration consistent with the property, not a proof. On Himmelblau, the damped method avoids the local maximum.

The price of Newton; on the computer

The price of curvature, per iteration

- $n_g = 1$, $n_H = 1$ ($2n^2 + 1$ evaluations of f if numerical, together);
- **dense** $n \times n$ system: $O(n^3)$ (sparse/structured: can be much less);
- **memory** $O(n^2)$ to store H — against $O(n)$ for steepest descent.

With millions of variables, classical Newton is not the natural choice.

Newton: **exact** curvature, expensive

quasi-Newton (BFGS): **approximate** curvature
from gradients

outside the syllabus; used by default in `fminunc`

The core, in MATLAB:

```
g = grad(x);  H = hess(x);  
d = -(H \ g);      % H*d = -g  
if g'*d >= 0  
    d = -g;        % safeguard  
end  
alpha = ...;      % line search  
x = x + alpha*d;
```

$[\sim, p] = \text{chol}(H)$: $p = 0$ if $H \succ 0$.

The full function (NewtonND, pure/damped, with `info.nfev`, `ngev`, `nhev`) is in the course companion code.

KEY TAKEAWAYS — 3.3.2

1. Newton locally minimizes the Taylor quadratic: $H_k \mathbf{d}_k = -\mathbf{g}_k$ (the system is solved, H_k is not inverted).
2. If f is an SPD quadratic, it reaches the minimum in **one iteration**.
3. Pure Newton seeks $\nabla f = \mathbf{0}$, **not minima**: if $H_k \neq 0$, the direction may not be a descent direction and the method may converge to maxima or saddles.
4. Pure Newton is local; in practice Hessian modification, safeguards and a line search are used. Near a minimum with $H(\mathbf{x}^*) \succ 0$, with a line search starting at $\alpha = 1$ (Armijo/Wolfe), $\alpha_k \rightarrow 1$ and, under regularity, **quadratic** convergence is recovered.
5. The price of curvature: Hessian ($O(n^2)$ memory) + dense system ($O(n^3)$).

Next (3.3.3): **can we avoid the zigzag of steepest descent without computing or storing the Hessian?** — conjugate gradients.

References Full list: *Course bibliography*, at the end of the course PDF.

- Nocedal, J., Wright, S. J., *Numerical Optimization*, 2nd ed., Springer, 2006 — Ch. 3.
- Dennis, J. E., Jr., Schnabel, R. B., *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, SIAM (Classics in Applied Mathematics 16), 1996 — Ch. 6.

Resources

- Companion code: NewtonND (pure/damped; `info.nfev`, `ngev`, `nhev`).
- MATLAB: `chol`; `fminunc` (quasi-Newton BFGS by default, outside the syllabus).